VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Rukith Nayak (1BM23CS277)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Rukith Nayak (1BM23CS277),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

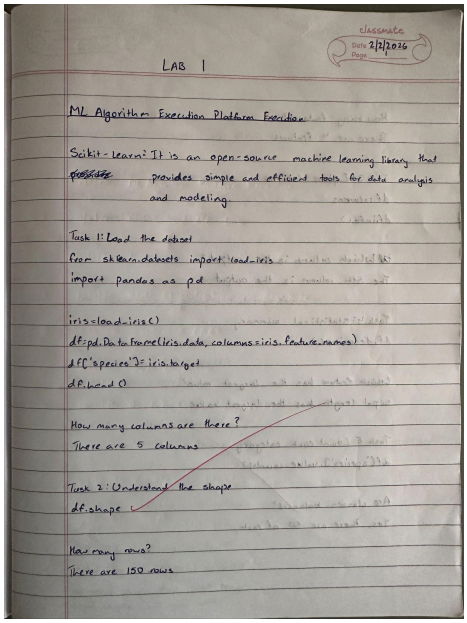
Saritha A. N Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Github Link:
https://github.com/RukithBMS/ML-1BM23CS277

Program 1

Screenshot



LAB 1

ML Algorithm Execution Platform Function

Scikit-learn: It is an open-source machine learning library that provides simple and efficient tools for data analysis and modeling.

Task 1: Load the dataset

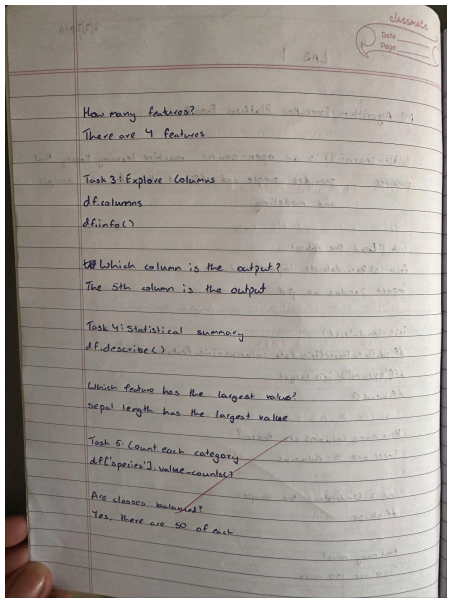
```
from sklearn.datasets import load_iris
import pandas as pd

iris=load_iris()
df=pd.DataFrame(iris.data, columns=iris.feature_names)
df['species']=iris.target
df.head()
```

How many columns are there?
There are 5 columns

Task 2: Understand the shape

```
df.shape
```

How many rows?
There are 150 rows



How many features?
There are 4 features

Task 3: Explore columns

```
df.columns
df.info()
```

Which column is the output?
The 5th column is the output

Task 4: Statistical summary

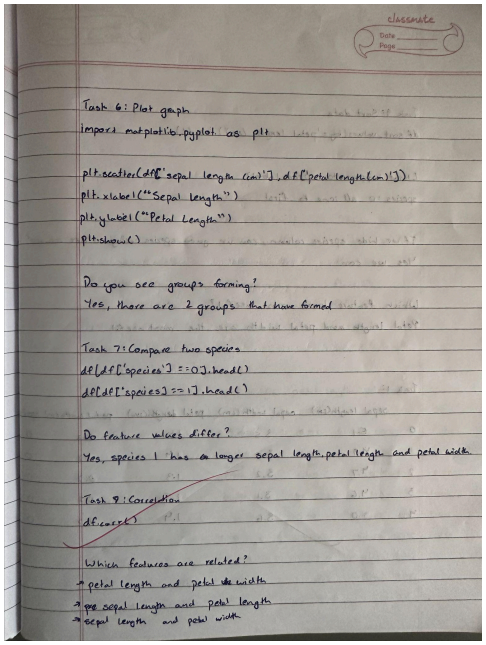
```
df.describe()
```

Which feature has the largest value?
Sepal length has the largest value

Task 5: Count each category

```
df['species'].value_counts()
```

Are classes balanced?
Yes, there are 50 of each



Task 6: Plot graph

```
import matplotlib.pyplot as plt

plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()
```

Do you see groups forming?
Yes, there are 2 groups that have formed

Task 7: Compare two species

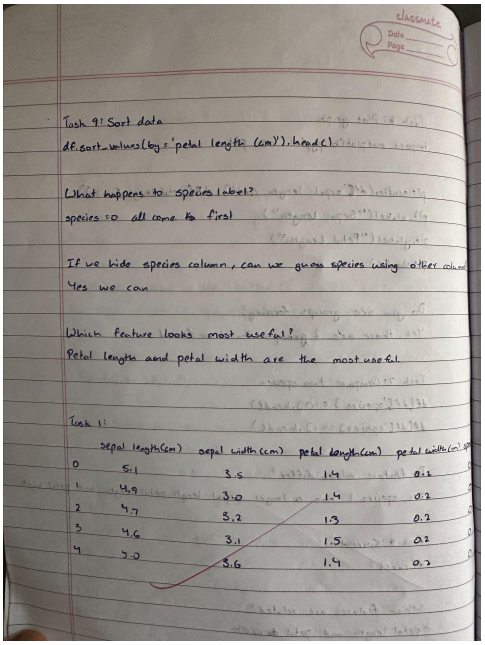
```
df[df['species'] == 0].head()
df[df['species'] == 1].head()
```

Do feature values differ?
Yes, species 1 has a longer sepal length, petal length and petal width

Task 8: Correlation

```
df.corr()
```

Which features are related?
→ petal length and petal width
→ sepal length and petal length
→ sepal length and petal width



Task 9: Sort data

```
df.sort_values(by='petal length (cm)').head()
```

What happens to species label?
species = 0 all come to first

If we hide species column, can we guess species using other columns?
Yes we can

Which feature looks most useful?
Petal length and petal width are the most useful

Task 1:

	sepal length(cm)	sepal width(cm)	petal length(cm)	petal width(cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Task 2:

(150, 5)

Task 3:

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal length	150 non-null	float64
1	sepal width	150 non-null	float64
2	petal length	150 non-null	float64
3	petal width	150 non-null	float64
4	species	150 non-null	float64

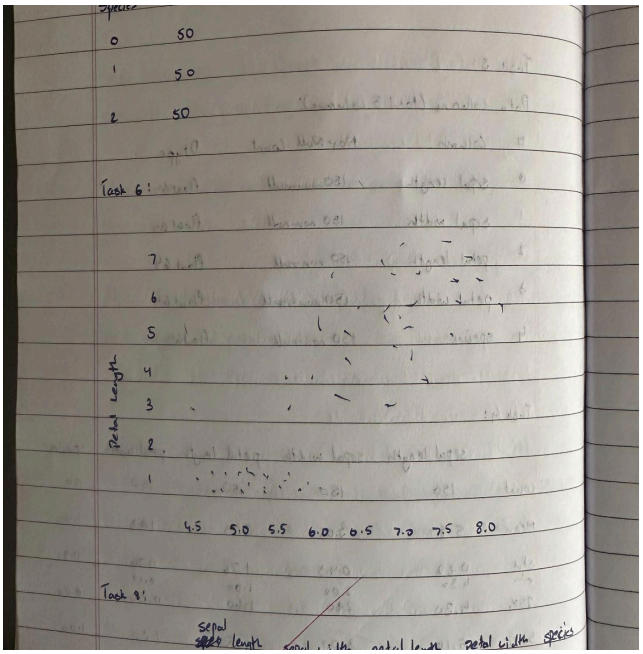
Task 4:

	sepal length	sepal width	petal length	petal width	species
count	150	150	150	150	150
mean	5.83	3.05	3.75	1.99	1.00
std	0.82	0.43	1.76	0.76	0.81
min	4.30	2.00	1.00	0.10	0.00
25%	4.30	2.80	1.60	0.30	0.00
50%	5.10	3.00	4.35	1.30	1.00
75%	5.80	3.30	5.10	1.80	2.00
max	7.70	4.40	6.90	2.50	2.00

Species

0	50
1	50
2	50

Task 6:



Task 8:

sepal length

Task 9:

	sepal length	sepal width	petal length	petal width	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Code:

In [12]:
```
from sklearn.datasets import load_iris
import pandas as pd
import matplotlib.pyplot as plt
```

In [6]:
```
iris = load_iris()
df =pd.DataFrame(iris.data,columns=iris["feature_names"])
df["species"] = iris.target
df.head()
```

Out[6]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

In [7]:
```
df.shape
```

Out[7]: (150, 5)

In [8]:
```
df.columns
```

Out[8]:
```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

In [9]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

In [10]:
```
df.describe()
```

Out[10]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

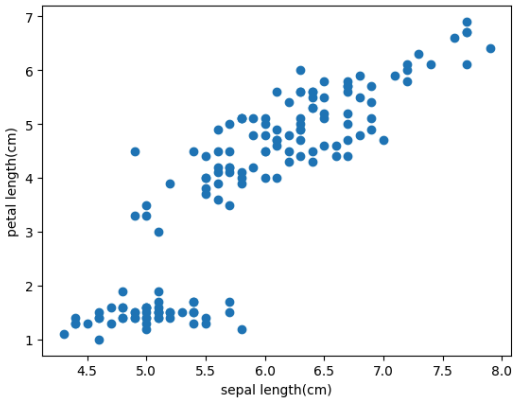
In [11]:
```
df['species'].value_counts()
```

Out[11]:

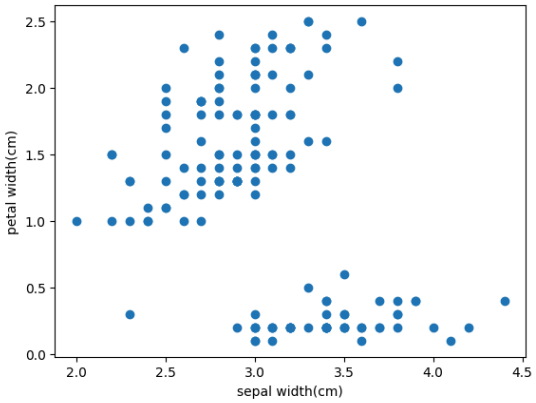
species	count
0	50
1	50
2	50

dtype: int64

In [13]:
```
plt.scatter(df['sepal length (cm)'],df['petal length (cm)'])
plt.xlabel('sepal length(cm)')
plt.ylabel('petal length(cm)')
plt.show()
```



In [15]:
```
plt.scatter(df['sepal width (cm)'],df['petal width (cm)'])
plt.xlabel('sepal width(cm)')
plt.ylabel('petal width(cm)')
plt.show()
```



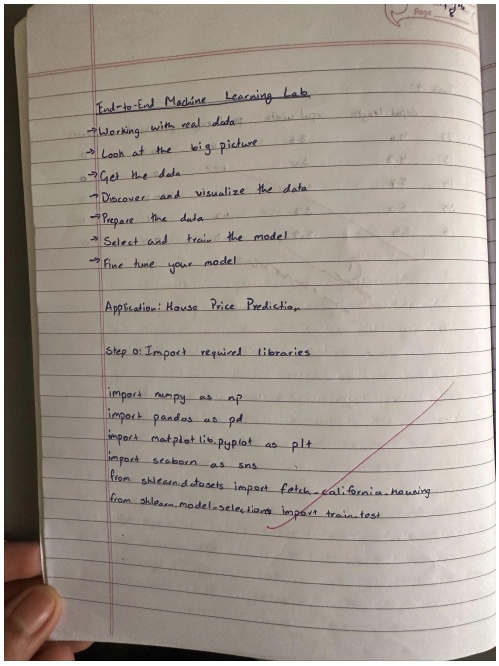
In [16]:
```
df[df.species==0].head()
```

Out[16]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Program 2

Screenshots:



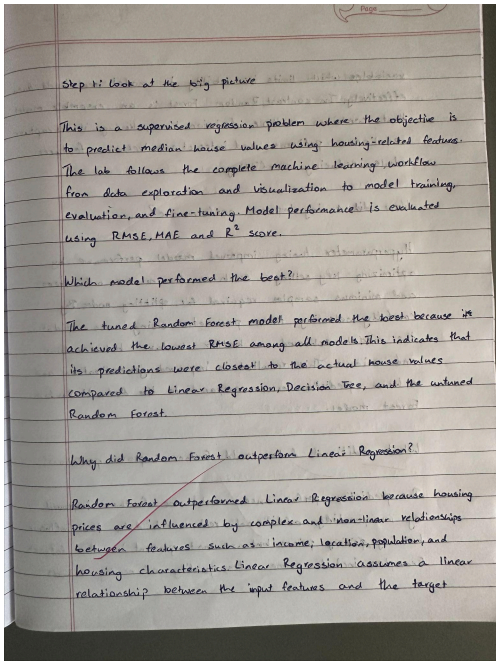
End-to-End Machine Learning Lab

→ Working with real data
→ Look at the big picture
→ Get the data
→ Discover and visualize the data
→ Prepare the data
→ Select and train the model
→ Fine tune your model

Application: House Price Prediction

Step 0: Import required libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test
```



Step 1: Look at the big picture

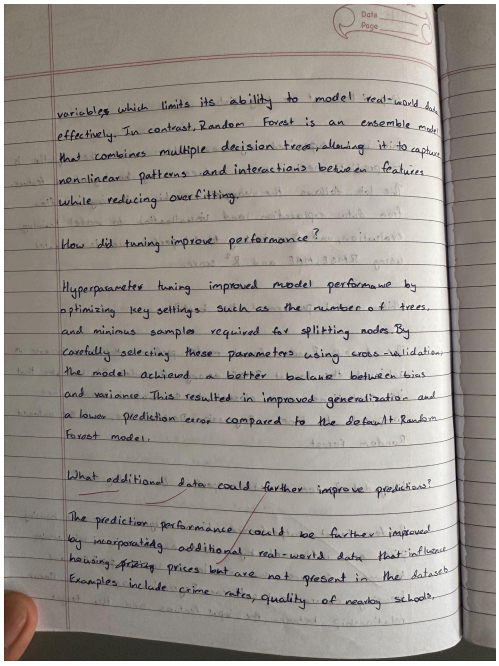
This is a supervised regression problem where the objective is to predict median house values using housing-related features. The lab follows the complete machine learning workflow from data exploration and visualization to model training, evaluation, and fine-tuning. Model performance is evaluated using RMSE, MAE and R^2 score.

Which model performed the best?

The tuned Random Forest model performed the best because it achieved the lowest RMSE among all models. This indicates that its predictions were closest to the actual house values compared to Linear Regression, Decision Tree, and the untuned Random Forest.

Why did Random Forest outperform Linear Regression?

Random Forest outperformed Linear Regression because housing prices are influenced by complex and non-linear relationships between features such as income, location, population, and housing characteristics. Linear Regression assumes a linear relationship between the input features and the target



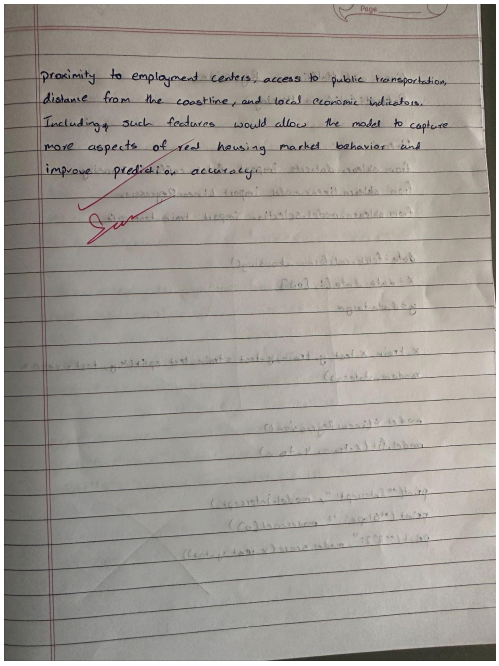
variables which limits its ability to model real-world data effectively. In contrast, Random Forest is an ensemble model that combines multiple decision trees, allowing it to capture non-linear patterns and interactions between features while reducing overfitting.

How did tuning improve performance?

Hyperparameter tuning improved model performance by optimizing key settings such as the number of trees, and minimum samples required for splitting nodes. By carefully selecting these parameters using cross-validation, the model achieved a better balance between bias and variance. This resulted in improved generalization and a lower prediction error compared to the default Random Forest model.

What additional data could further improve predictions?

The prediction performance could be further improved by incorporating additional real-world data that influence housing pricing prices but are not present in the dataset. Examples include crime rates, quality of nearby schools,



proximity to employment centers, access to public transportation, distance from the coastline, and local economic indicators. Including such features would allow the model to capture more aspects of real housing market behavior and improve prediction accuracy.

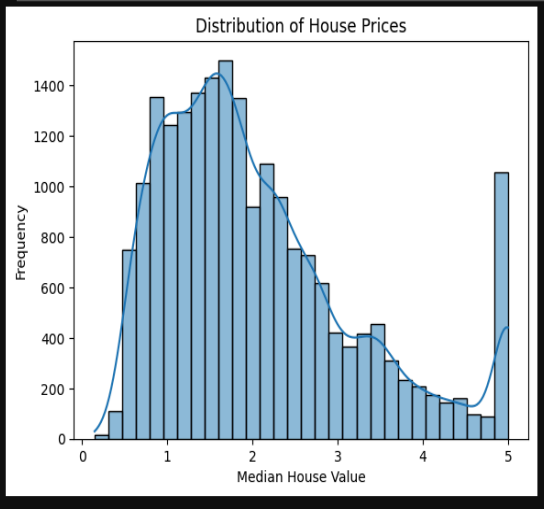
Code:

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

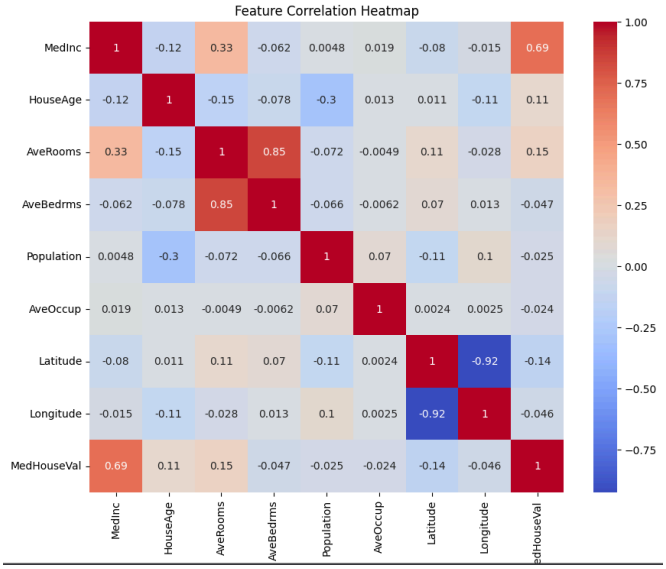
In [ ]:
```
# Load California housing dataset
housing = fetch_california_housing(as_frame=True)

# Create DataFrame
df = housing.frame

# Display basic info
df.head()
df.shape
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   MedInc       20640 non-null  float64
 1   HouseAge     20640 non-null  float64
 2   AveRooms     20640 non-null  float64
 3   AveBedrms    20640 non-null  float64
 4   Population   20640 non-null  float64
 5   AveOccup     20640 non-null  float64
 6   Latitude     20640 non-null  float64
 7   Longitude    20640 non-null  float64
 8   MedHouseVal  20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```

In [ ]:
```
plt.figure()
sns.histplot(df['MedHouseVal'], bins=30, kde=True)
plt.xlabel("Median House Value")
plt.ylabel("Frequency")
plt.title("Distribution of House Prices")
plt.show()
```

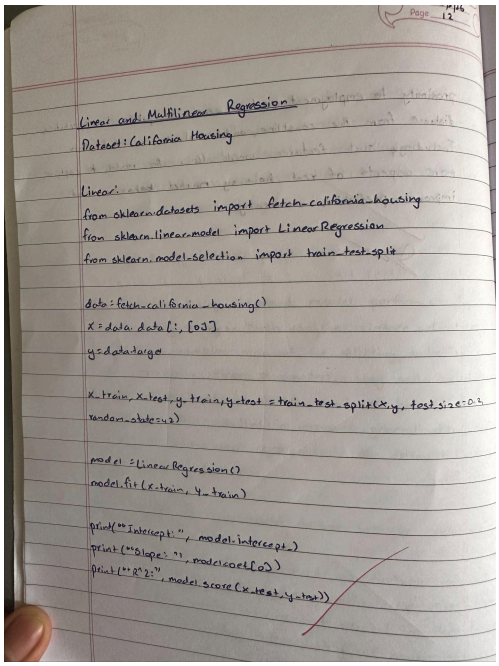


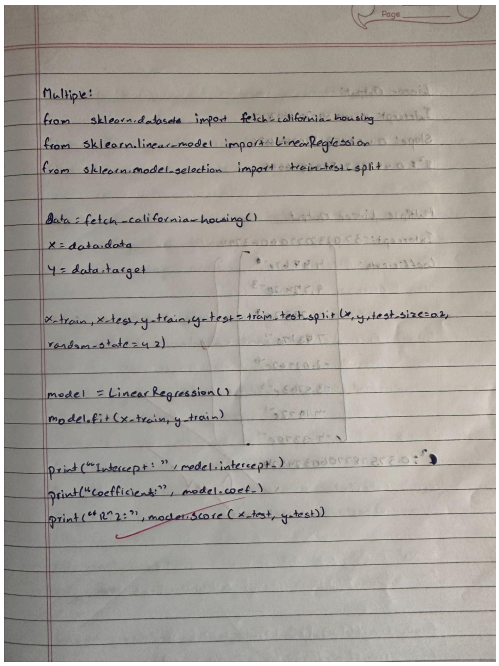
In [ ]:
```
plt.figure()
sns.scatterplot(x=df['MedInc'], y=df['MedHouseVal'], alpha=0.5)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Income vs House Price")
plt.show()
```

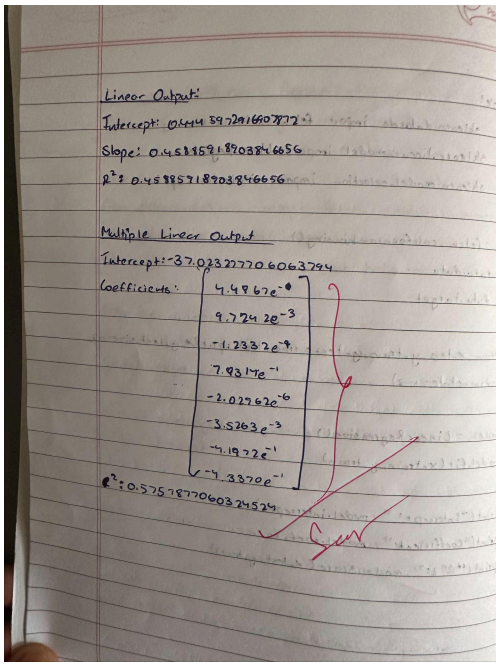


Program 3 & 4

Screenshot







Code:

```
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
# Load dataset
data = fetch_california_housing()
X = data.data[:, [0]]   # Use only MedInc (first feature)
y = data.target

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train
model = LinearRegression()
model.fit(X_train, y_train)

print("Intercept:", model.intercept_)
print("Slope:", model.coef_[0])
print("R^2:", model.score(X_test, y_test))
```

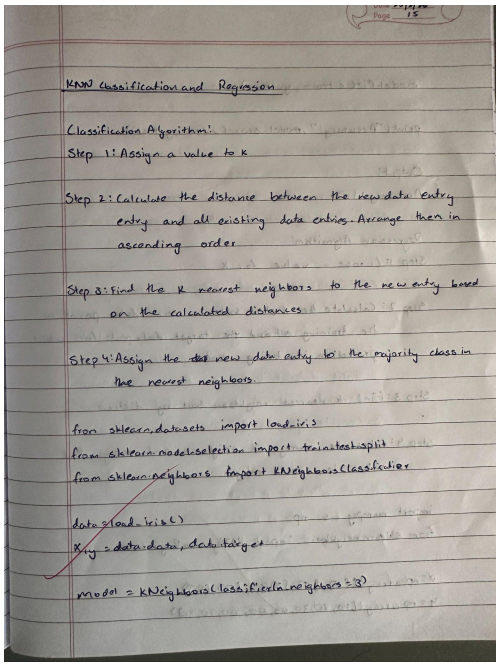
O/P:
Intercept: 0.4445972916907872
Slope: 0.4193384939381273
R^2: 0.45885918903846656

```
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Load dataset
data = fetch_california_housing()
X = data.data   # All features
y = data.target

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train
model = LinearRegression()
model.fit(X_train, y_train)

print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
print("R^2:", model.score(X_test, y_test))
```

O/P:
Intercept: -37.023277706063794
Coefficients: [4.48674910e-01 9.72425752e-03 -1.23323343e-01 7.83144907e-01
 -2.02962058e-06 -3.52631849e-03 -4.19792487e-01 -4.33708065e-01]
R^2: 0.5757877060324524

Program 5

Screenshot



KNN Classification and Regression

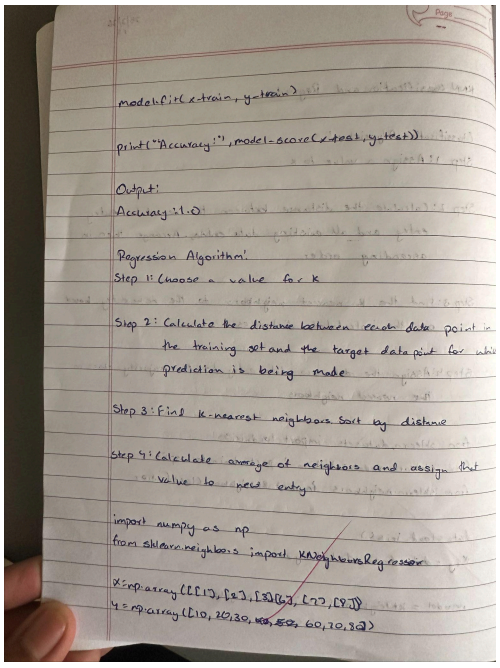
Classification Algorithm:
Step 1: Assign a value to k

Step 2: Calculate the distance between the new data entry and all existing data entries. Arrange them in ascending order

Step 3: Find the k nearest neighbors to the new entry based on the calculated distances

Step 4: Assign the new data entry to the majority class in the nearest neighbors

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

data = load_iris()
X, y = data.data, data.target

model = KNeighborsClassifier(classifier__neighbors = 3)
```



```
model.fit(x_train, y_train)

print("Accuracy:", model.score(x_test, y_test))
```

Output:
Accuracy: 1.0

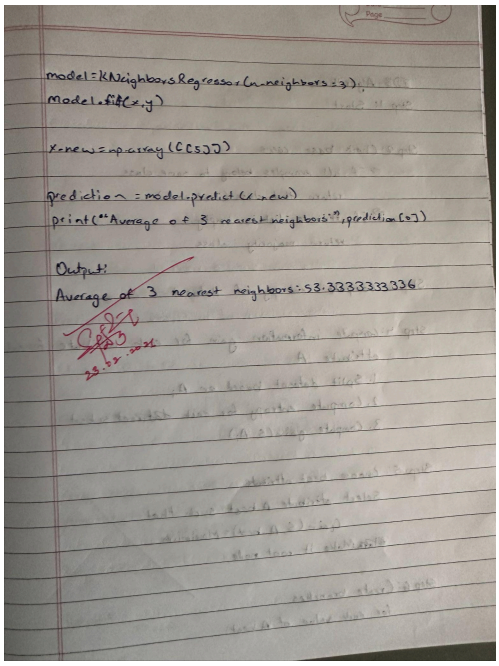
Regression Algorithm:
Step 1: Choose a value for k

Step 2: Calculate the distance between each data point in the training set and the target data point for which prediction is being made

Step 3: Find k-nearest neighbors. Sort by distance

Step 4: Calculate average of neighbors and assign that value to new entry

```
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

X = np.array([[1], [2], [3], [6], [7], [8]])
y = np.array([10, 20, 30, 50, 60, 70, 80])
```



```
model = KNeighborsRegressor(n_neighbors = 3)
model.fit(X, y)

X_new = np.array([[5]])

prediction = model.predict(X_new)
print("Average of 3 nearest neighbors:", prediction[0])
```

Output:
Average of 3 nearest neighbors: 53.33333333336

Code:

```
import numpy as np
```

```
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
X = np.array([[100], [105], [98], [150], [160], [155]])
y_class = np.array([0, 0, 0, 1, 1, 1])
y_reg = np.array([100, 105, 98, 150, 160, 155])
# 1) KNN Classification
knn_classifier = KNeighborsClassifier(n_neighbors=3)
knn_classifier.fit(X, y_class)
new_ball = np.array([[110]])
class_prediction = knn_classifier.predict(new_ball)
if class_prediction[0] == 0:
    print("Classification: Red Ball")
else:
    print("Classification: Blue Ball")
# 2) KNN Regression
knn_regressor = KNeighborsRegressor(n_neighbors=3)
knn_regressor.fit(X, y_reg)
weight_prediction = knn_regressor.predict(new_ball)
print("Regression (Average Predicted Weight):", weight_prediction[0])
```

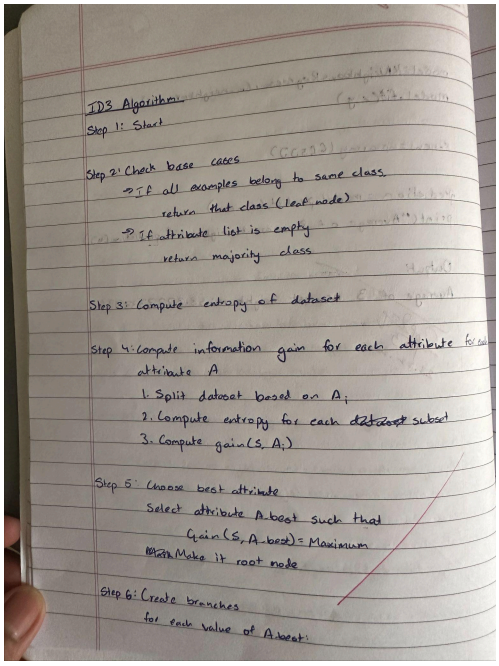
O/P:

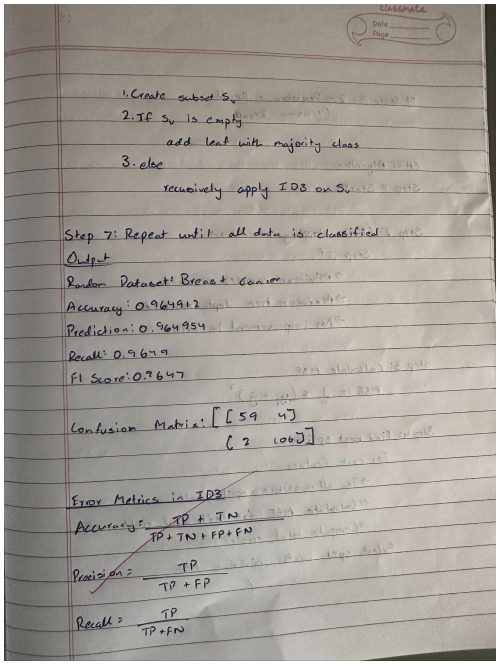
Classification: Red Ball

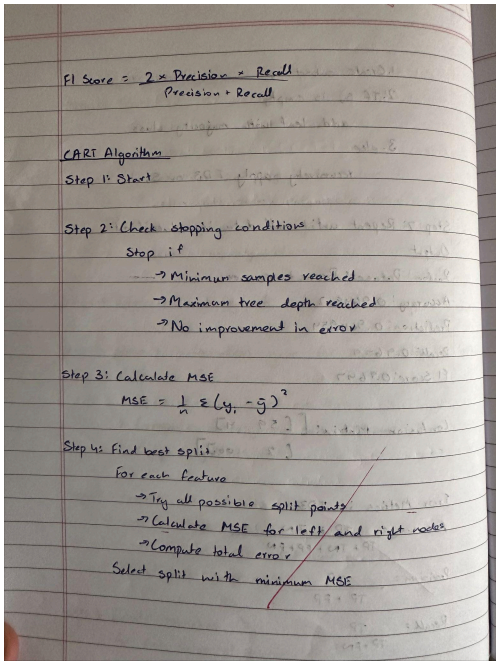
Regression (Average Predicted Weight): 101.0

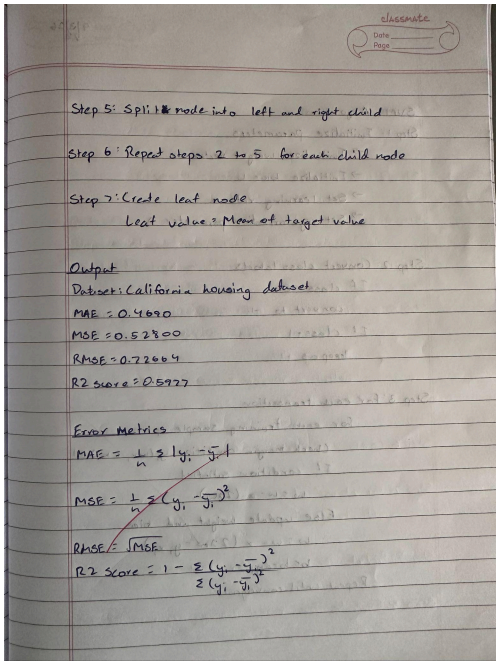
Program 6

Screenshots:









Code:

```
import numpy as np
```

```
import pandas as pd
from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
def evaluate_classification(X, y, dataset_name):
    print("\n=====================================")
    print("Dataset:", dataset_name)
    print("=====================================")
# Split dataset
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
    )
 # ID3 → entropy criterion
    model = DecisionTreeClassifier(criterion="entropy", random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
 # Error Metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')
    cm = confusion_matrix(y_test, y_pred)
    print("Accuracy:", accuracy)
```

```
    print("Precision:", precision)
    print("Recall:", recall)
    print("F1 Score:", f1)
    print("\nConfusion Matrix:")
    print(cm)
# Dataset 1: Iris
iris = load_iris()
evaluate_classification(iris.data, iris.target, "Iris Dataset")
# Dataset 2: Breast Cancer
cancer = load_breast_cancer()
evaluate_classification(cancer.data, cancer.target, "Breast Cancer Dataset")
```

O/P:

```
==================================
Dataset: Iris Dataset
==================================
Accuracy: 0.9777777777777777
Precision: 0.9793650793650793
Recall: 0.9777777777777777
F1 Score: 0.9777448559670783

Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  1 12]]
```

==========

Dataset: Breast Cancer Dataset

==========

Accuracy: 0.9649122807017544

Precision: 0.9649541140481607

Recall: 0.9649122807017544

F1 Score: 0.9647902680643604

Confusion Matrix:

[[59 4]

 [2 106]]

```
# Decision Tree Regression using CART Algorithm
import numpy as np
from sklearn.datasets import fetch_california_housing, load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
def evaluate_regression(X, y, dataset_name):
    print("\n==========")
    print("Dataset:", dataset_name)
    print("==========")
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.3, random_state=42
    )
# CART → squared_error criterion
    model = DecisionTreeRegressor(
        criterion="squared_error",
        random_state=42
    )
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
 # Error Metrics
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
    print("MAE:", mae)
    print("MSE:", mse)
    print("RMSE:", rmse)
    print("R2 Score:", r2)
# Dataset 1: California Housing
housing = fetch_california_housing()
evaluate_regression(housing.data, housing.target, "California Housing Dataset")
# Dataset 2: Diabetes
diabetes = load_diabetes()
evaluate_regression(diabetes.data, diabetes.target, "Diabetes Dataset")
```

O/P:

==========

Dataset: California Housing Dataset

==========

MAE: 0.46904822189922485

MSE: 0.5280096503174904

RMSE: 0.7266427253592307

R2 Score: 0.5977192261218356

==========

Dataset: Diabetes Dataset

==========

MAE: 60.015037593984964

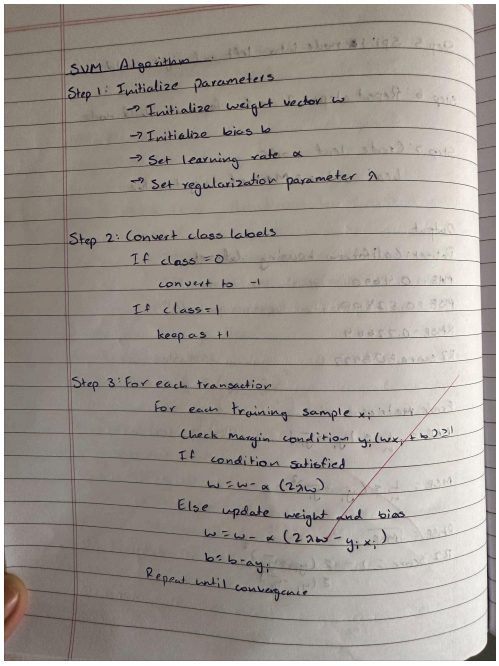
MSE: 5697.789473684211

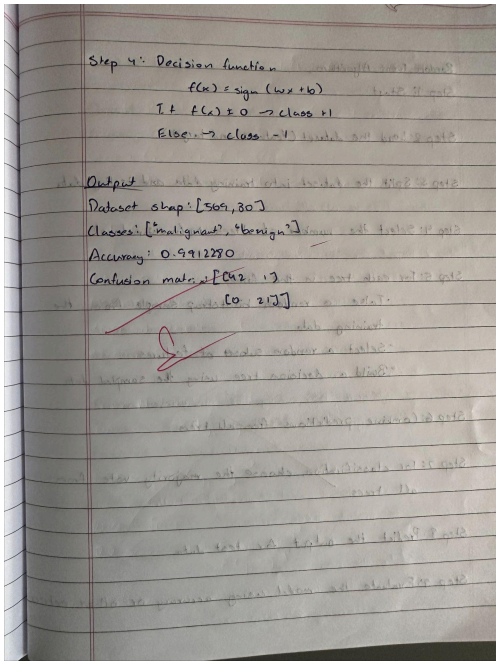
RMSE: 75.48370336492647

R2 Score: -0.05547696148647652

Program 7

Screenshots:





Code:

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Create SVM model
svm_model = SVC(kernel='linear')
# Train model
svm_model.fit(X_train, y_train)
# Predictions
y_pred = svm_model.predict(X_test)
# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

O/P:

```
Accuracy: 0.9666666666666667
Confusion Matrix
[[10  0  0]
 [ 0  8  1]
```

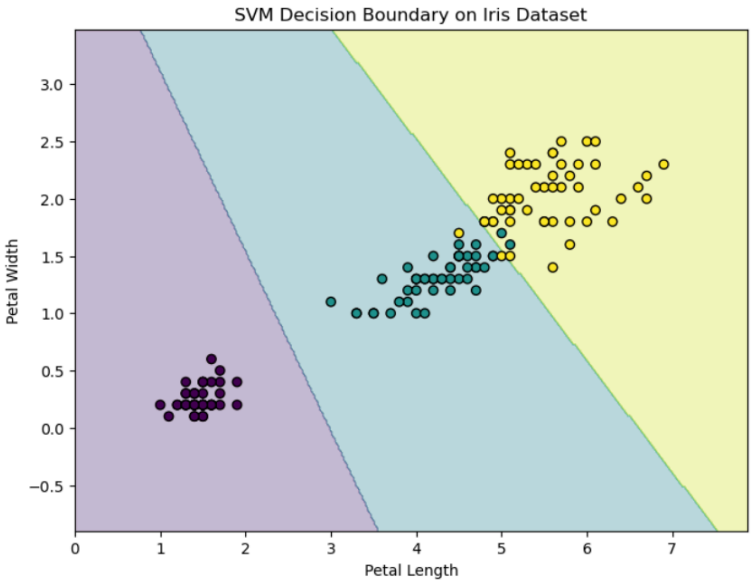
[0 0 11]]

Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

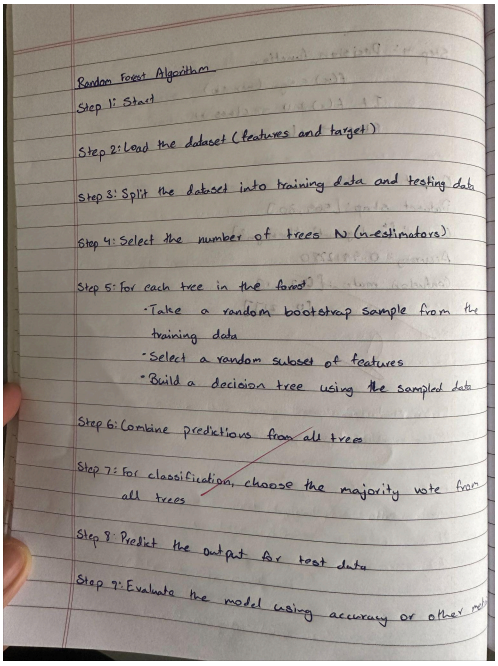
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
iris = datasets.load_iris()
X = iris.data[:, 2:4]   # petal length and petal width
y = iris.target
model = SVC(kernel='linear')
model.fit(X, y)
SVC(kernel='linear')
x_min, x_max = X[:,0].min() - 1, X[:,0].max() + 1
y_min, y_max = X[:,1].min() - 1, X[:,1].max() + 1
```

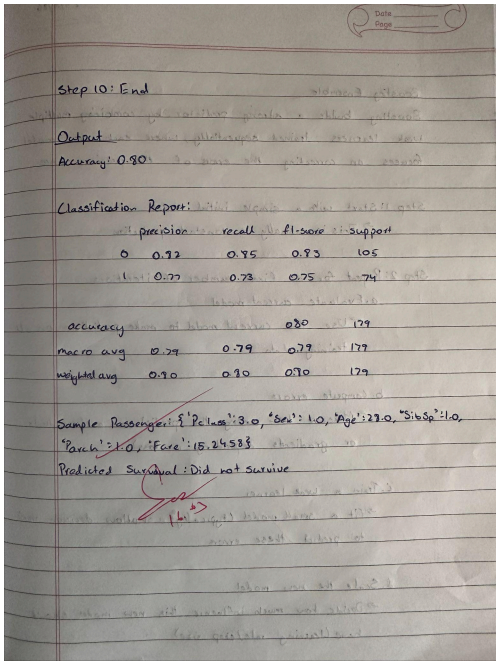
```
xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.02),
    np.arange(y_min, y_max, 0.02)
)
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.figure(figsize=(8,6))
plt.contourf(xx, yy, Z, alpha=0.3)
scatter = plt.scatter(X[:,0], X[:,1], c=y, edgecolors='k')
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("SVM Decision Boundary on Iris Dataset")
plt.show()
```



Program 8

Screenshots





Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report

import warnings

warnings.filterwarnings('ignore')

# Load dataset

titanic_data = pd.read_csv('titanic.csv')

# Remove rows where Survived is missing

titanic_data = titanic_data.dropna(subset=['Survived'])

# Select features and target
```

```
X = titanic_data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']].copy()
y = titanic_data['Survived']
# Encode categorical variable
X['Sex'] = X['Sex'].map({'female': 0, 'male': 1})
# Fill missing Age values with median
X['Age'] = X['Age'].fillna(X['Age'].median())
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create Random Forest model
rf_classifier = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
# Train model
rf_classifier.fit(X_train, y_train)
# Make predictions
y_pred = rf_classifier.predict(X_test)
# Evaluate model
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:\n", classification_rep)
```

```
# Test with a sample passenger
sample = X_test.iloc[[0]]
prediction = rf_classifier.predict(sample)
sample_dict = sample.iloc[0].to_dict()
print("\nSample Passenger:", sample_dict)
print("Predicted Survival:", "Survived" if prediction[0] == 1 else "Did Not Survive")
```

Accuracy: 0.80

Classification Report:

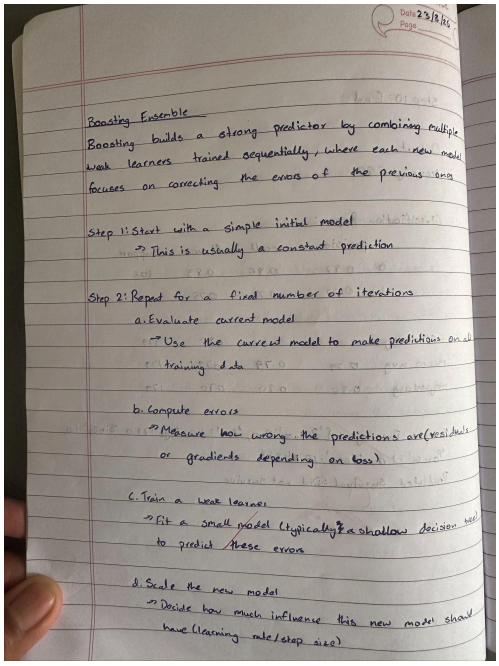
	precision	recall	f1-score	support
0	0.82	0.85	0.83	105
1	0.77	0.73	0.75	74
accuracy			0.80	179
macro avg	0.79	0.79	0.79	179
weighted avg	0.80	0.80	0.80	179

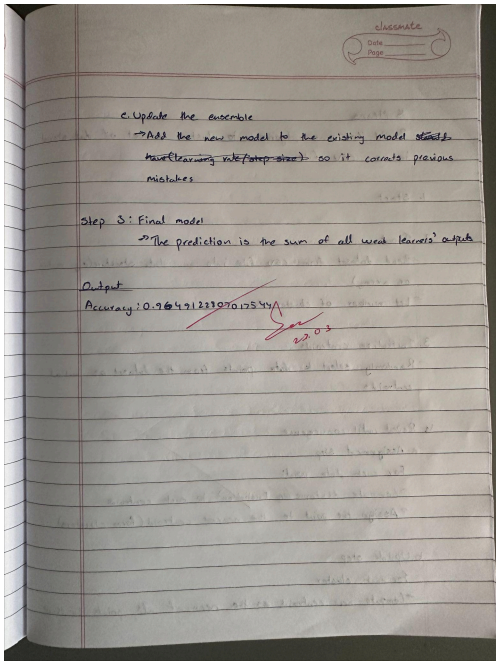
Sample Passenger: {'Pclass': 3.0, 'Sex': 1.0, 'Age': 28.0, 'SibSp': 1.0, 'Parch': 1.0, 'Fare': 15.2458}

Predicted Survival: Did Not Survive

Program 9

Screenshots





Code:

 # Step 1: Import libraries

import numpy **as** np

import pandas **as** pd

import matplotlib.pyplot **as** plt

from sklearn.datasets **import** load_breast_cancer

from sklearn.model_selection **import** train_test_split

from sklearn.ensemble **import** AdaBoostClassifier

from sklearn.tree **import** DecisionTreeClassifier

from sklearn.decomposition **import** PCA

from sklearn.metrics **import** accuracy_score, classification_report, confusion_matrix

 # Step 2: Load dataset

```
data = load_breast_cancer()
X = data.data
y = data.target
# Step 3: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Step 4: Create base learner
base_model = DecisionTreeClassifier(max_depth=1)
# Step 5: Create AdaBoost model
model = AdaBoostClassifier(
    estimator=base_model,
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)
# Step 6: Train model
model.fit(X_train, y_train)
# Step 7: Predictions
y_pred = model.predict(X_test)
# Step 8: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

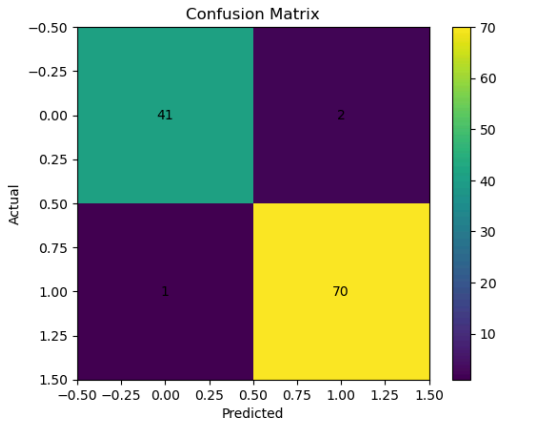
```
# ------------------------------
# 🔷 VISUALIZATION 1: Confusion Matrix Heatmap
# ------------------------------
cm = confusion_matrix(y_test, y_pred)
plt.figure()
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j, i, cm[i, j], ha='center', va='center')
plt.colorbar()
plt.show()
# ------------------------------
# 🔷 VISUALIZATION 2: Error Reduction Plot
# ------------------------------
errors = []
for y_pred_stage in model.staged_predict(X_test):
    errors.append(np.mean(y_pred_stage != y_test))
plt.figure()
plt.plot(errors)
plt.title("Error Reduction over Boosting Iterations")
plt.xlabel("Number of Estimators")
```

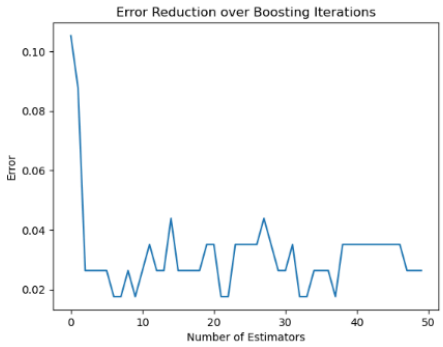
```
plt.ylabel("Error")
plt.show()
# ------------------------------
# 🔷 VISUALIZATION 3: PCA Decision Boundary
# ------------------------------
# Reduce to 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
# Train again on PCA data
model_pca = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
    random_state=42
)
model_pca.fit(X_pca, y)
# Plot boundary
x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1
y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200)
)
Z = model_pca.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

```
plt.figure()
plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
plt.title("AdaBoost Decision Boundary (PCA Reduced Data)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```

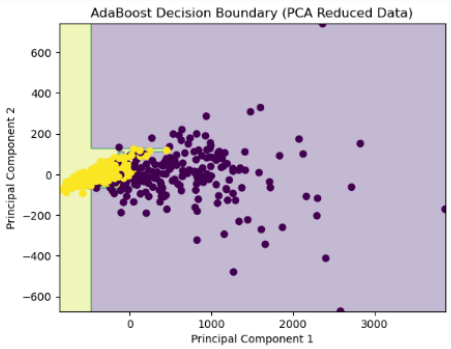
O/P:

```
Accuracy: 0.9736842105263158

Confusion Matrix:
 [[41  2]
 [ 1 70]]

Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.95      0.96        43
           1       0.97      0.99      0.98        71

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\ensemble\_weight_boosting.p
  warnings.warn(
```



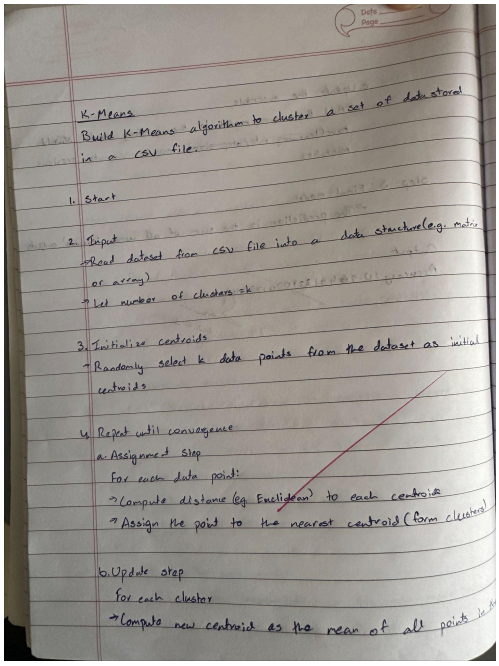


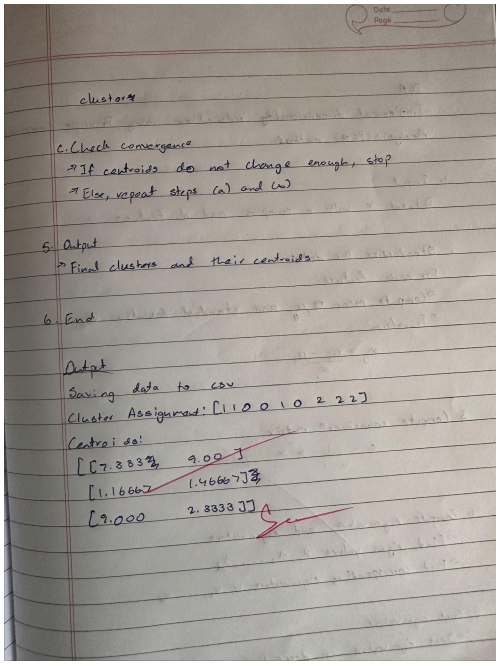

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\ensemble\_weight_boosting.py:527: FutureWarning:
  warnings.warn(
```



Program 10

Screenshots





Code:

```
import pandas as pd

import numpy as np

from google.colab import files


# Load CSV data

def load_data(file_path):

    data = pd.read_csv(file_path)

    return data.values  # convert to numpy array


# Initialize centroids randomly
```

```
def initialize_centroids(data, k):
    indices = np.random.choice(len(data), k, replace=False)
    return data[indices]


# Assign clusters
def assign_clusters(data, centroids):
    clusters = []
    for point in data:
        distances = [np.linalg.norm(point - centroid) for centroid in centroids]
        cluster = np.argmin(distances)
        clusters.append(cluster)
    return np.array(clusters)


# Update centroids
def update_centroids(data, clusters, k):
    new_centroids = []
    for i in range(k):
        points = data[clusters == i]
        if len(points) == 0:
            new_centroids.append(data[np.random.randint(0, len(data))])
        else:
            new_centroids.append(np.mean(points, axis=0))
    return np.array(new_centroids)
```

```
# K-Means algorithm
def kmeans(file_path, k, max_iters=100, tol=1e-4):
    data = load_data(file_path)
    centroids = initialize_centroids(data, k)

    for _ in range(max_iters):
        clusters = assign_clusters(data, centroids)
        new_centroids = update_centroids(data, clusters, k)

        # Check convergence
        if np.linalg.norm(new_centroids - centroids) < tol:
            break

        centroids = new_centroids

    return clusters, centroids

# Example usage
if __name__ == "__main__":
    uploaded = files.upload()
    file_path = "data.csv"  # replace with your CSV file
    k = 3

    clusters, centroids = kmeans(file_path, k)
```

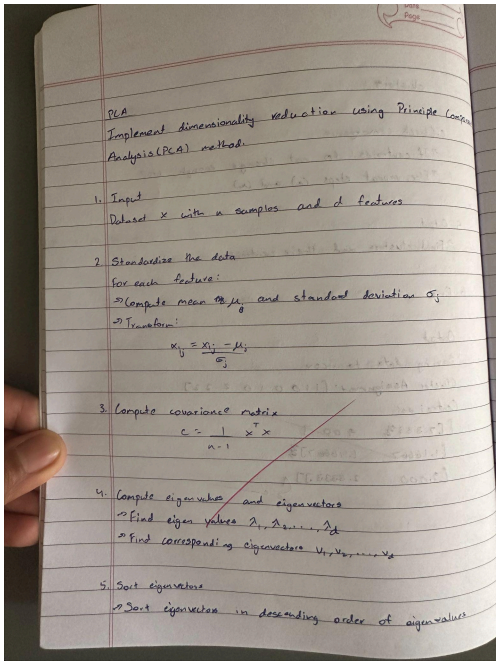
```
print("Cluster assignments:", clusters)

print("Centroids:\n", centroids)
```

O/P:

```
Saving data.csv to data.csv
Cluster assignments: [1 1 0 0 1 0 2 2 2]
Centroids:
 [[7.33333333 9.        ]
 [1.16666667 1.46666667]
 [9.         2.33333333]]
```

Program 11

Screenshots


PCA
Implement dimensionality reduction using Principle Component Analysis (PCA) method.

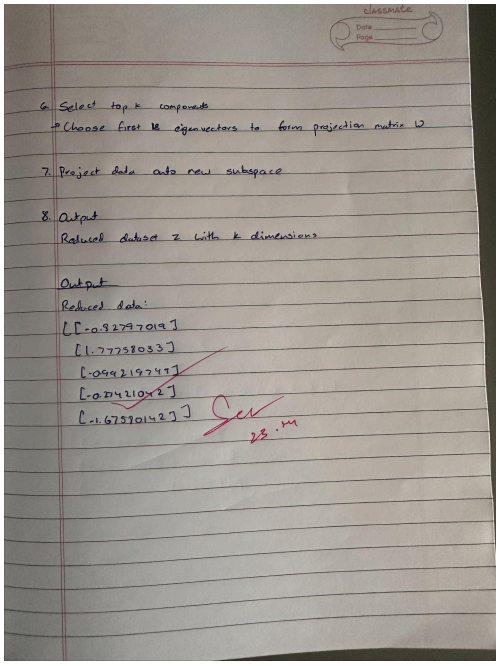
1. Input
Dataset x with n samples and d features

2. Standardize the data
For each feature:
→ Compute mean μ_j and standard deviation σ_j
→ Transform:
$$x_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

3. Compute covariance matrix
$$C = \frac{1}{n-1} X^T X$$

4. Compute eigenvalues and eigenvectors
→ Find eigen values $\lambda_1, \lambda_2, \ldots, \lambda_d$
→ Find corresponding eigenvectors $v_1, v_2, \ldots, v_d$

5. Sort eigenvectors
→ Sort eigenvectors in descending order of eigenvalues


6. Select top k components
→ Choose first k eigenvectors to form projection matrix W

7. Project data onto new subspace

8. Output
Reduced dataset z with k dimensions

Output
Reduced data:
[[-0.92797019]
[1.77758033]
[-0.99219749]
[-0.87422039]
[-1.67539142]]
23.11

Code and O/P:

In [1]:

```
from sklearn.decomposition import PCA
import pandas as pd

# Load CSV
data = pd.read_csv("wine_dataset.csv")

# Apply PCA (reduce to 2 dimensions)
pca = PCA(n_components=2)
reduced_data = pca.fit_transform(data)

print("Reduced Shape:", reduced_data.shape)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

```
Reduced Shape: (178, 2)
Explained Variance Ratio: [0.99809123 0.00173592]
```

In [2]:

```
import matplotlib.pyplot as plt

plt.scatter(reduced_data[:, 0], reduced_data[:, 1])
plt.title("PCA - Wine Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```